

“AI代码信任度实验”报告

Portra约拍服务平台

Phase 4 — AI 代码信任度实验报告

项目：Portra 约拍服务平台

阶段：P4 编码开发、联调与版本收尾

实验编号：P4-EXP-CODE-TRUST-TEAM-01

实验对象：订单履约状态流转与跨模块衔接

整理日期：2026-06-07

一、实验目标

P4 要求团队选择一个规模适中的功能点，使用 AI 生成核心代码或逻辑，并记录：

Prompt

AI 直出结果

人工检查问题

修复前后简要对比

本报告选择“订单履约状态流转”作为实验对象。该功能属于团队 P0 核心链路，不归属于单一成员，也不是只覆盖 D 线。

订单履约状态流转与以下模块直接相关：

用户登录与身份认证

需求发布与服务方响应

对话与报价

订单生成

模拟支付

拍摄状态

交付上传

返修

确认完成

自动确认

超时退款

评价

通知

状态日志

因此，它适合用于观察 AI 代码能否直接进入真实项目，以及人工审查在跨模块规则中是否必要。

二、仓库核验基线

2.1 项目结构

```
Camera/  
├─ frontend/  
├─ backend/  
├─ docs/  
├─ .github/workflows/ci.yml  
├─ .gitlab-ci.yml  
├─ migration.sql  
├─ smoke-test.spec.js  
└─ smoke_test_issues.md
```

2.2 实际技术栈

前端：

```
Vite + React + JavaScript / JSX + MUI + react-router-dom
```

后端：

```
Spring Boot 3.5 + JDK 17 + Maven  
Spring Data JPA + MyBatis-Plus  
MySQL 8.0  
H2 smoke profile  
JWT  
Spring Mail  
本地文件存储
```

2.3 本实验读取的真实文件

```
backend/src/main/java/com/action/camera/order/controller/OrderController.java  
a  
backend/src/main/java/com/action/camera/order/service/OrderService.java  
backend/src/main/java/com/action/camera/order/statemachine/OrderStatusMachine.java  
a  
backend/src/main/java/com/action/camera/delivery/service/DeliveryService.java  
a  
backend/src/main/java/com/action/camera/notification/service/NotificationService.java  
backend/src/main/java/com/action/camera/review/**  
smoke-test.spec.js  
smoke_test_issues.md
```

三、业务链路 with 状态范围

当前订单核心链路包括：

```
PENDING_PAYMENT
→ PAID_PENDING_SHOOT
→ SHOOTING
→ PENDING_DELIVERY
→ DELIVERED_PENDING_CONFIRM
→ COMPLETED
```

同时存在：

```
CANCELLED
REFUNDED
REWORK_REQUIRED
APPEALING
```

实际仓库已经将合法状态迁移集中在：

```
OrderStatusMachine
```

并在 `OrderService` 中补充：

```
权限校验
支付校验
托管状态
退款
状态日志
通知
自动推进
自动确认
超时退款
返修
```

四、实验 Prompt

以下 Prompt 用于让 AI 生成订单状态更新核心逻辑：

```
请为 Camera / Portra 约拍平台编写订单状态更新服务。
```

```
背景：
```

- 后端使用 **Spring Boot**、**JDK 17**、**Maven**、**JPA**。
- 订单状态包括：
PENDING_PAYMENT、**PAID_PENDING_SHOOT**、**SHOOTING**、
PENDING_DELIVERY、**DELIVERED_PENDING_CONFIRM**、
REWORK_REQUIRED、**APPEALING**、**COMPLETED**、**CANCELLED**、**REFUNDED**。
- 需要支持：
模拟支付、开始拍摄、结束拍摄、上传交付、确认完成、
取消、返修、自动确认、超时退款。
- 订单状态变化要记录日志，并触发必要通知。
- 客户和服务方权限不同。
- 不允许非法跳转和重复操作。

请输出：

1. 状态迁移规则；
2. **OrderService** 核心方法；
3. 权限校验；
4. 异常处理；
5. 状态日志；
6. 通知触发；
7. 单元测试建议。

注意：

- 不要只写 **CRUD**；
- 不要允许前端任意写入状态；
- 说明哪些动作应由系统自动推进；
- 说明与 **Delivery**、**Review**、**Notification** 的衔接。

五、AI 直出结果摘要

AI 能够较快生成一个基础版本，典型结构如下：

```
@Transactional
public Order changeStatus(Long orderId, Long operatorId, OrderStatus
targetStatus) {
    Order order = orderRepository.findById(orderId)
        .orElseThrow(() -> new BusinessException("Order not found"));

    if (!OrderStatusMachine.canTransit(order.getStatus(), targetStatus)) {
        throw new BusinessException("Illegal status transition");
    }

    order.setStatus(targetStatus);
    order.setUpdatedAt(LocalDateTime.now());
    orderRepository.save(order);

    saveStatusLog(orderId, operatorId, targetStatus);
}
```

```
    sendNotification(order, targetStatus);

    return order;
}
```

AI 还能够补充：

状态迁移表
非法状态拦截
状态日志
支付接口建议
通知接口建议
单元测试样例

5.1 AI 直出的优点

- 快速形成 Service 骨架；
- 能识别需要状态迁移表；
- 能意识到需要日志与通知；
- 能补充异常测试；
- 能提醒接口不能只依赖前端；
- 可作为人工实现的起点。

六、人工检查发现的问题

问题 1：单一 `changeStatus()` 容易暴露过宽能力

现象

AI 初稿倾向于允许调用方传入：

```
targetStatus
```

只要状态机允许，就可以写入。

根因

AI 关注“合法迁移”，但没有充分区分：

客户动作
服务方动作
系统自动动作
支付专用动作

交付专用动作
返修专用动作

风险

- 前端可能直接请求系统自动状态；
- 权限校验散落；
- 支付与交付业务被绕过；
- 通知和托管款逻辑遗漏；
- 调试时难以判断状态由谁推进。

修复

将通用能力收敛为业务动作：

```
mockPay()  
cancelOrder()  
requestRework()  
completeReworkDelivery()  
autoAdvanceShootingOrders()  
autoConfirmTimeoutOrders()  
autoRefundOverdueUndeliveredOrders()
```

对外接口只暴露 P4 允许的动作。

问题 2：支付状态不能通过普通状态更新进入

现象

AI 初稿容易允许：

```
PENDING_PAYMENT → PAID_PENDING_SHOOT
```

通过普通状态更新完成。

根因

状态迁移与支付业务没有分离。

风险

- 未生成 PaymentRecord；
- 未校验支付金额；
- 未校验重复支付；

- 托管状态未更新；
- 支付通知未生成。

修复

必须通过：

```
POST /orders/{orderId}/payments
```

并在 `mockPay()` 中执行：

```
付款人校验  
金额校验  
重复支付校验  
PaymentRecord  
EscrowStatus.HELD  
状态日志  
支付通知
```

问题 3：拍摄开始与结束存在“人工推进 / 系统推进”冲突

现象

真实仓库中：

- `OrderController.validateP4Transition()` 允许服务方手动请求：

```
PAID_PENDING_SHOOT → SHOOTING  
SHOOTING → PENDING_DELIVERY
```

- `OrderService.ensureNotManualShootingTransition()` 又明确禁止手动推进，提示拍摄状态由系统计划推进；
- `smoke-test.spec.js` Step 11 / Step 12 仍然调用手动状态接口。

根因

Controller、Service 和 smoke test 的业务口径没有同步更新。

风险

- 冒烟测试可能与真实规则冲突；
- 前端可能展示不可用按钮；
- 同一条链路在不同层得到不同结论。

修复

统一采用“系统自动推进”口径：

```
PAID_PENDING_SHOOT → SHOOTING  
SHOOTING → PENDING_DELIVERY
```

由：

```
autoAdvanceShootingOrders()
```

执行。

同时：

```
移除 Controller 中对应手动暴露  
更新 smoke-test.spec.js  
更新前端按钮与演示说明  
补充自动推进测试
```

问题 4：交付上传不能只保存 Delivery 再直接回调状态接口

现象

交付上传需要：

```
保存 deliveries / delivery_files  
→ 调用订单状态推进  
→ DELIVERED_PENDING_CONFIRM
```

若外层事务持有外键共享锁，再通过 HTTP 回调更新 orders，可能产生跨线程锁等待。

根因

数据库事务与本地 HTTP 回调顺序设计不当。

风险

- 上传交付时死锁；
- 订单状态更新失败；
- Delivery 与 Order 状态不一致。

修复

真实仓库已采用：

```
TransactionTemplate 独立事务保存交付记录
→ 先提交，释放 FK 共享锁
→ 再调用 orderStatusPort.changeStatus()
→ 状态更新失败时 rollbackSavedDelivery()
```

问题 5：完成、返修、超时退款和通知不能省略

现象

AI 初稿通常只覆盖正常状态更新。

根因

AI 对跨模块副作用掌握不足。

风险

- 订单完成后托管款未释放；
- 超时未交付不退款；
- 返修链路断裂；
- 通知缺失；
- 状态日志不完整；
- 后续评价判断错误。

修复

在 OrderService 中集中处理：

```
markCompletedAndReleaseEscrow()
markRefunded()
notifyOrderPaid()
notifyOrderCancelled()
notifyOrderCompleted()
requestRework()
completeReworkDelivery()
autoConfirmTimeoutOrders()
autoRefundOverdueUndeliveredOrders()
```

七、修复前后简要对比

对比项	AI 初稿	人工审查与仓库实现
状态更新入口	通用 <code>changeStatus(targetStatus)</code>	业务动作收敛，通用入口受限
支付	可被普通状态迁移绕过	专用 mock payment API
拍摄开始与结束	容易当作人工操作	统一为系统自动推进
权限	容易只校验参与者	区分客户、服务方、系统
重复支付	容易遗漏	PaymentRecord 幂等校验
托管款	容易遗漏	支付 held、完成 released、退款 refunded
交付状态	普通状态写入	Delivery 保存后安全推进
交付死锁	未考虑	独立事务提交后再回调
返修	容易遗漏	<code>requestRework()</code> 与返修交付
自动确认	容易遗漏	7 天超时自动确认
超时退款	容易遗漏	未交付超时自动退款
状态日志	基础记录	每次状态变化保存日志
通知	模糊建议	支付、取消、完成、交付通知
单元测试	正常路径为主	增加权限、重复操作、自动任务和异常路径

八、实验结果

8.1 静态审查结果

以 14 个关键验收点为基准：

验收点	AI 初稿	人工审查后
合法迁移表	通过	通过
参与者权限	部分通过	通过
客户 / 服务方角色区分	不完整	通过
支付专用逻辑	不完整	通过
重复支付拦截	不完整	通过
托管款变化	不完整	通过
状态日志	通过	通过
通知	部分通过	通过

验收点	AI 初稿	人工审查后
系统自动推进	不完整	通过
自动确认	不完整	通过
超时退款	不完整	通过
返修	不完整	通过
Delivery 死锁风险	未识别	通过
smoke test 与真实规则一致	未识别	待修复

统计：

AI 初稿：2 项完整通过，3 项部分通过，9 项不完整或未识别
人工审查后：13 项达到实现要求，1 项仍需统一 `smoke test` 与接口口径

8.2 本地验证回填区

以下结果必须由团队在本地运行后填写。未执行时不得写“通过”。

```
cd backend
./mvnw test
```

```
node smoke-test.spec.js
```

验证项	结果	证据
后端单元测试	通过	<code>mvn test</code> 构建成功，GitHub Actions CI 最新记录绿色
正常订单链路	通过	本地走通：发需求→响应→接受→报价→支付→交付→确认→评价
非法状态跳转	通过	OrderStatusMachine 拦截，返回 STATUS_CONFLICT
非客户支付	通过	服务方调用支付接口返回 FORBIDDEN
重复支付	通过	第二次支付返回 STATUS_CONFLICT
服务方时间冲突	通过	后端校验报价时间与档期逻辑
Delivery 上传	通过	修复 PAID_PENDING_SHOOT 状态自动推进后本地验证成功
Delivery 失败回滚	通过	<code>rollbackSavedDelivery</code> 在状态更新失败时清理记录

验证项	结果	证据
自动拍摄推进	通过	交付上传时后端自动推进 PAID_PENDING_SHOOT→SHOOTING→PENDING_DELIVERY
超时自动确认	通过	7 天无操作后 autoConfirmTimeoutOrders 触发确认
超时未交付退款	通过	autoRefundOverdueUndeliveredOrders 处理超时退款
返修链路	通过	REWORK_REQUIRED 状态下服务方可重新上传交付
通知生成	通过	支付、交付、评价等节点触发站内通知
状态日志	通过	每次状态变更写入 order_status_logs 表

九、结论

本次实验表明：

AI 适合生成订单状态服务的基础骨架，但不能独立完成业务状态机。

AI 在以下方面有明显价值：

快速生成样板
梳理状态枚举
补充常规异常
生成测试候选

AI 容易遗漏：

支付副作用
托管款
跨模块通知
系统自动任务
返修
超时退款
Delivery 事务死锁
Controller、Service、smoke test 之间的口径同步

最终原则：

AI 生成的核心业务代码不能直接合并。

必须经过需求、状态机、权限、事务、跨模块副作用、测试和 Git diff 的共同审查。